

NSL-KDD-Dataset Preprocessing

Import Data

```

1 import pandas as pd
2 from google.colab import drive
3 drive.mount('/content/drive')
4 df = pd.read_csv('/content/drive/MyDrive/CyberSecurityAIDataset/NSL-KDD-Datas

```

Check Data

```

1 print(df.shape)
2 print(df.head())
3 print(df[41].value_counts())

```

```

(125973, 43)
   0    1    2    3    4    5    6    7    8    9    ...    33    34    35  \
0  0  tcp  ftp_data  SF  491    0    0    0    0    0  ...  0.17  0.03  0.17
1  0  udp   other  SF  146    0    0    0    0    0  ...  0.00  0.60  0.88
2  0  tcp  private  S0    0    0    0    0    0  ...  0.10  0.05  0.00
3  0  tcp   http  SF  232  8153    0    0    0    0  ...  1.00  0.00  0.03
4  0  tcp   http  SF  199   420    0    0    0    0  ...  1.00  0.00  0.00

```

```

   36    37    38    39    40    41    42
0  0.00  0.00  0.00  0.05  0.00  normal  20
1  0.00  0.00  0.00  0.00  0.00  normal  15
2  0.00  1.00  1.00  0.00  0.00  neptune  19
3  0.04  0.03  0.01  0.00  0.01  normal  21
4  0.00  0.00  0.00  0.00  0.00  normal  21

```

```
[5 rows x 43 columns]
```

```

41
normal          67343
neptune         41214
satan           3633
ipsweep         3599
portsweep       2931
smurf           2646
nmap            1493
back            956
teardrop        892
warezclient     890
pod             201
guess_passwd    53
buffer_overflow 30
warezmaster     20
land            18
imap            11
rootkit         10
loadmodule      9
ftp_write       8
multihop        7
phf             4
perl            3
spy             2
Name: count, dtype: int64

```

Edit Column Names and remove difficulty score column

```

1 columns = [
2     'duration','protocol_type','service','flag','src_bytes','dst_bytes',
3     'land','wrong_fragment','urgent','hot','num_failed_logins','logged_in',
4     'num_compromised','root_shell','su_attempted','num_root','num_file_creati
5     'num_shells','num_access_files','num_outbound_cmds','is_host_login',
6     'is_guest_login','count','srv_count','serror_rate','srv_serror_rate',
7     'rerror_rate','srv_rerror_rate','same_srv_rate','diff_srv_rate',
8     'srv_diff_host_rate','dst_host_count','dst_host_srv_count',
9     'dst_host_same_srv_rate','dst_host_diff_srv_rate','dst_host_same_src_port
10    'dst_host_srv_diff_host_rate','dst_host_serror_rate','dst_host_srv_serror
11    'dst_host_rerror_rate','dst_host_srv_rerror_rate','label','difficulty'
12 ]
13
14 df.columns = columns
15 df = df.drop('difficulty', axis=1)

```

Simplify attack types

```

1 df['label'] = df['label'].apply(lambda x: 0 if x== 'normal' else 1)
2 # 0=normal, 1=attack

```

Encode categorical columns

```

1 from sklearn.preprocessing import LabelEncoder
2
3 le = LabelEncoder()
4 for col in ['protocol_type', 'service', 'flag']:
5     df[col] = le.fit_transform(df[col])

```

Split features and label

```

1 X = df.drop('label', axis=1).values
2 y= df['label'].values

```

Normalize

```

1 from sklearn.preprocessing import StandardScaler
2
3 scaler = StandardScaler()
4 X = scaler.fit_transform(X)

```

Train/Test split

```

1 from sklearn.model_selection import train_test_split
2 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, ranc

```

Convert To Tensor

```
1 import torch
2
3 X_train = torch.FloatTensor(X_train)
4 X_test = torch.FloatTensor(X_test)
5 y_train = torch.FloatTensor(y_train).unsqueeze(1)
6 y_test = torch.FloatTensor(y_test).unsqueeze(1)
```

```
1 print(X_train.shape)
2 print(y_train.shape)
```

```
torch.Size([100778, 41])
torch.Size([100778, 1])
```

Create Pytorch Model

```
1 import torch
2 from torch import nn
3
4 device = "cuda" if torch.cuda.is_available() else "cpu"
5 device
```

```
'cuda'
```

```
1 import torch
2 import torch.nn as nn
3
4 class IntrusionDetector(nn.Module):
5     def __init__(self):
6         super().__init__()
7         self.network = nn.Sequential(
8             nn.Linear(41, 64),
9             nn.ReLU(),
10            nn.Linear(64, 32),
11            nn.ReLU(),
12            nn.Linear(32, 1),
13            nn.Sigmoid()
14        )
15
16    def forward(self, x):
17        return self.network(x)
18
19 model = IntrusionDetector()
20 print(model)
```

```
IntrusionDetector(
  (network): Sequential(
    (0): Linear(in_features=41, out_features=64, bias=True)
    (1): ReLU()
    (2): Linear(in_features=64, out_features=32, bias=True)
    (3): ReLU()
    (4): Linear(in_features=32, out_features=1, bias=True)
    (5): Sigmoid()
```

```
)
)
```

Training

```
1 criterion = nn.BCELoss()
2 optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
3
4 epochs = 20
5 for epoch in range(epochs):
6     model.train()
7     optimizer.zero_grad()
8     outputs = model(X_train)
9     loss = criterion(outputs, y_train)
10    loss.backward()
11    optimizer.step()
12
13    if(epoch+1) % 5 == 0:
14        print(f'Epoch {epoch+1}/{epochs}, Loss: {loss.item():.4f}')
```

```
Epoch 5/20, Loss: 0.6541
Epoch 10/20, Loss: 0.6196
Epoch 15/20, Loss: 0.5778
Epoch 20/20, Loss: 0.5277
```

Testing

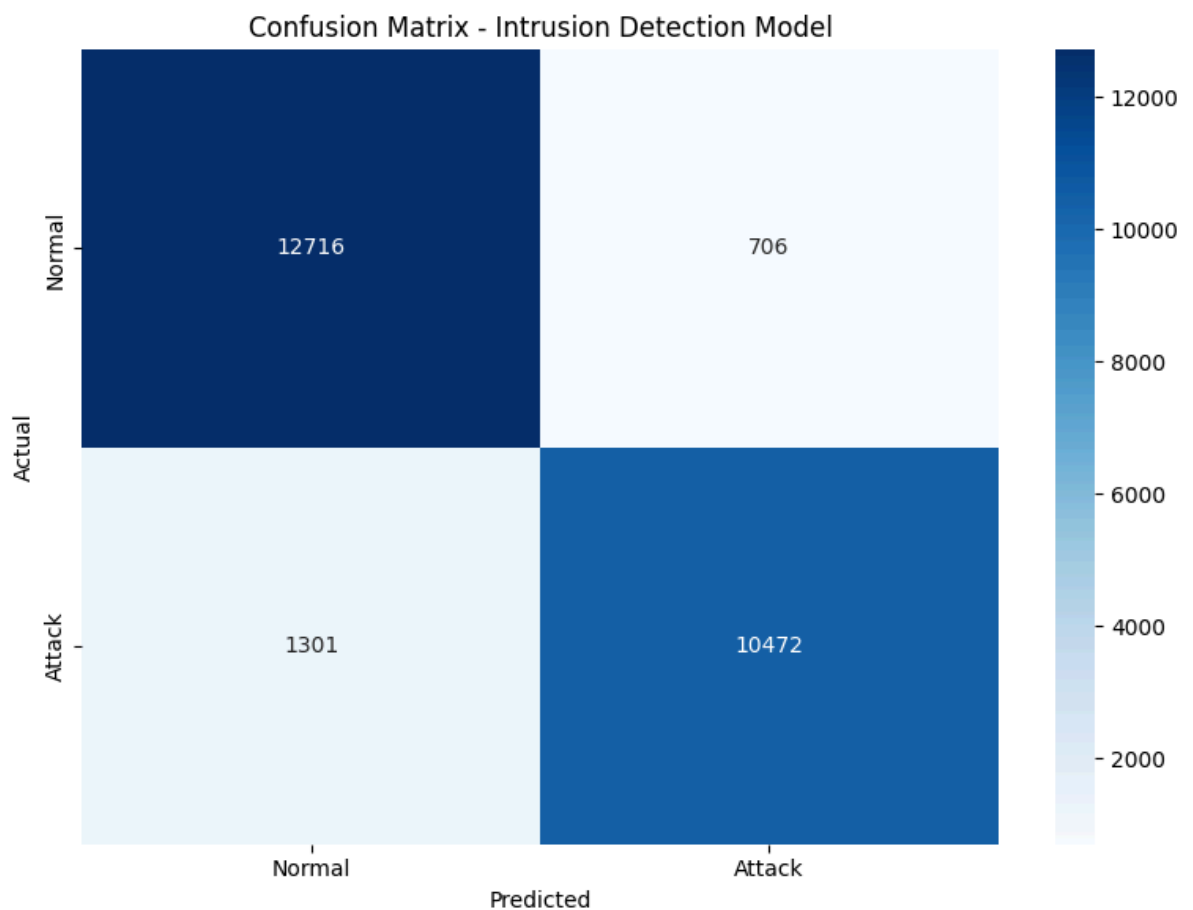
```
1 from sklearn.metrics import classification_report, confusion_matrix
2 import numpy as np
3
4 model.eval()
5 with torch.no_grad():
6     predictions = model(X_test)
7     predicted_labels = (predictions >= 0.5).float()
8
9 y_true = y_test.numpy()
10 y_pred = predicted_labels.numpy()
11
12 print(classification_report(y_true, y_pred, target_names=['Normal', 'Attack'])
13 print("Confusion Matrix:")
14 print(confusion_matrix(y_true, y_pred))
15
16
```

	precision	recall	f1-score	support
Normal	0.91	0.95	0.93	13422
Attack	0.94	0.89	0.91	11773
accuracy			0.92	25195
macro avg	0.92	0.92	0.92	25195
weighted avg	0.92	0.92	0.92	25195

```
Confusion Matrix:
[[12716  706]
 [ 1301 10472]]
```

Visualize

```
1 import matplotlib.pyplot as plt
2 import seaborn as sns
3
4 cm = confusion_matrix(y_true, y_pred)
5
6 plt.figure(figsize=(8,6))
7 sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
8             xticklabels=['Normal', 'Attack'],
9             yticklabels=['Normal', 'Attack'])
10 plt.title('Confusion Matrix - Intrusion Detection Model')
11 plt.ylabel('Actual')
12 plt.xlabel('Predicted')
13 plt.tight_layout()
14 plt.savefig('confusion_matrix.png')
15 plt.show()
```



1 Start coding or [generate](#) with AI.

